



Guided Sequential Decision Making

Decision-Time and Demonstration-Based Planning

CS4246/CS5446

Reinforcement Learning and Sequential Decision Making

Version 1.0



Topics

- Decision-Time (Online) Planning
 - Rollout Online Search
 - Monte Carlo Tree Search
 - Upper Confidence Bounds applied to Trees
- Demonstration-Based (Offline) Planning
 - Imitation Learning

Handling Curse of Dimensionality

When full planning or full reinforcement learning (RL) training is Infeasible

- **Decision-time planning:**
 - Online search at runtime
- **Demonstration-based:**
 - Learn from expert data
- **Model-free RL:**
 - Function approximation, policy search
- **Model-based RL:**
 - Learn or use world models (may include human priors)
- **Large Foundation Models (LFMs)-assisted and Human-Guided Planning :**
 - Model creation, task decomposition, heuristics, plan generation

Motivation

- When Full Planning or Full RL Training is Infeasible:

Decision-Time Planning	Demonstration-Based Planning
When: At runtime	When: Before runtime
How: Simulate ahead via online search (e.g., Rollout, Monte Carlo Tree Search)	How: Learn from expert demonstrations (e.g., Imitation Learning, Inverse Reinforcement Learning)
Goal: Choose next best action from current state	Goal: Pre-learn safe and effective policies

Recall: Markov Decision Process (MDP)

- Formally:

- An MDP $M \triangleq (S, A, T, R, \gamma)$ consists of:
- A set S of states
- A set A of actions
- A transition function $T: S \times A \times S \rightarrow [0,1]$ that satisfies the **Markov property** such that:

$$\forall s \in S, \forall a \in A: \sum_{s' \in S} T(s, a, s') = \sum_{s' \in S} P(s'|s, a) = 1$$

- A reward function $R: S \rightarrow \mathbb{R}$ or $R: S \times A \rightarrow \mathbb{R}$ or $R: S \times A \times S \rightarrow \mathbb{R}$
- A discount factor $0 \leq \gamma \leq 1$
- Solution is a policy – a function to recommend an action in each state: $\pi: S \rightarrow A$
 - Solution involves careful balancing of risk and reward



Decision-Time Planning

Online Search at Runtime



Main Approach

- Large Markov decision process (MDP):
 - State space grows exponentially
 - Offline methods (Value Iteration, Policy Iteration) are infeasible
- Solution:
 - Search forward (build search tree) only from current state
 - Plan just enough to pick the next good action
 - Avoid full offline policy computation



Recall: Q-Function

- Q-Function (Action-Utility Function)

- State utility $U(s)$ is defined by the Bellman Optimality Equation.
- To select actions, it is useful to evaluate state–action pairs directly.

- Definition (Bellman Optimality Equation for Q):

$$Q(s, a) = \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma U(s')]$$

- State utility: $U(s) = \max_a Q(s, a)$
- Optimal policy: $\pi^*(s) = \operatorname{argmax}_a Q(s, a)$

- Interpretation:

- $Q(s, a)$ = expected utility of taking action a in state s , then following the optimal policy.

Notes: Utility of States and State Sequences

Quantity	Meaning	Over what
$U(\langle s_t, s_{t+1}, \dots \rangle) = G_t$	Actual discounted reward sum (return)	One realised state sequence
$U^\pi(s_t) = \mathbb{E}[G_t \mid S_t = s_t]$	Expected utility from state s_t under policy π	All sequences from s_t
$U(s_t) = U^{\pi^*}(s_t)$	Optimal expected utility from state s_t	All sequences from s_t following π^*
$Q^\pi(s_t, a_t) = \mathbb{E}[G_t \mid S_t = s_t, A_t = a_t]$	Expected utility from (s_t, a_t) under π	All sequences from (s_t, a_t)

$$\boxed{
 \begin{aligned}
 U(\langle s_t, s_{t+1}, \dots \rangle) = G_t &\implies U^\pi(s_t) = \mathbb{E}[G_t \mid S_t = s_t] \implies U(s_t) = \max_{\pi} U^\pi(s_t) \\
 Q^\pi(s_t, a_t) &= \mathbb{E}[G_t \mid S_t = s_t, A_t = a_t]
 \end{aligned}
 }$$

Rollout Algorithms

- For a given state s :
 - For each action a , simulate many trajectories using rollout policy π
 - Estimate:

$$Q^\pi(s, a) \approx \text{average return}$$

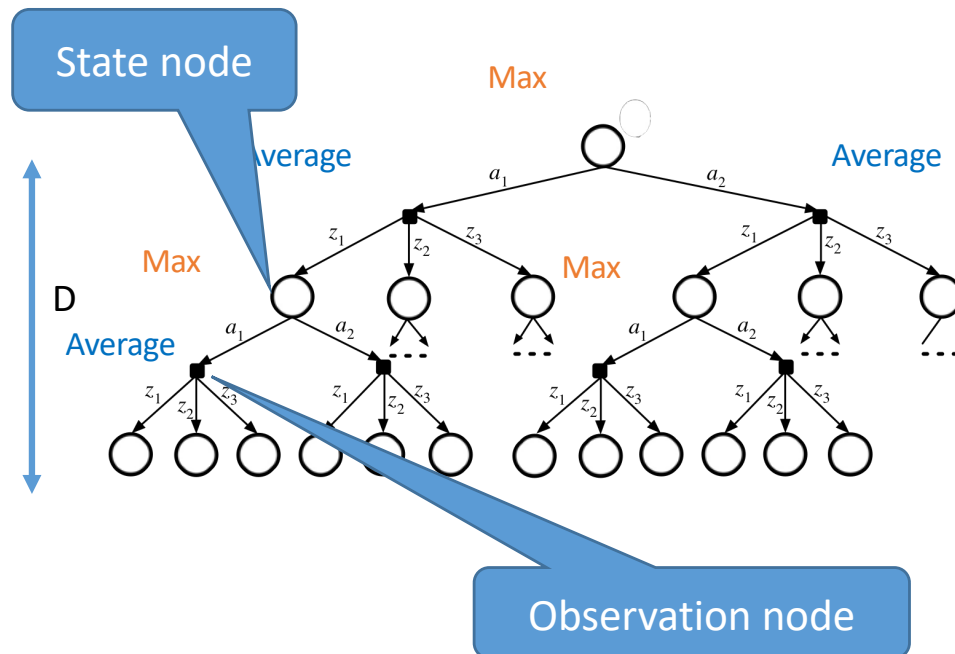
- Decision:

$$a^* = \arg \max_a Q^\pi(s, a)$$

- Insight:

- One-step *policy improvement* at decision time
- No need to compute full utility (value) function

Rollout Online Search



Decision-Time and Demo-
Based Planning

- Build search tree from current state (depth D):
 - State nodes (circles):
Agent actions $\rightarrow$ take max
 - Observation nodes (squares):
Environment outcomes $\rightarrow$ take average
 - Leaves: Utility from sampled rollouts
- Backup:
 - Average $\rightarrow$ Max $\rightarrow$ Root $\rightarrow$ pick best action

Persy the Mars Rover Adventures ...

- Persy is at a junction (s_0) must choose to go Left or Right.
 - For each action, the rollout algorithm simulates several random future trajectories using a simple policy π and records their returns.
 - It averages these returns to estimate $Q^\pi(s_0, a)$ for each action, then picks the action with the highest estimated return.
- This gives:
 - *One-step lookahead* decision-making without computing the full value function.

Scenario

One-step decision at s_0 :

- Actions:
 - a_L = go Left, a_R = go Right.
- Rollout policy π : random thereafter; horizon $H = 2$.

$$\hat{Q}^\pi(s_0, a) = \frac{1}{N} \sum_{i=1}^N G_i$$

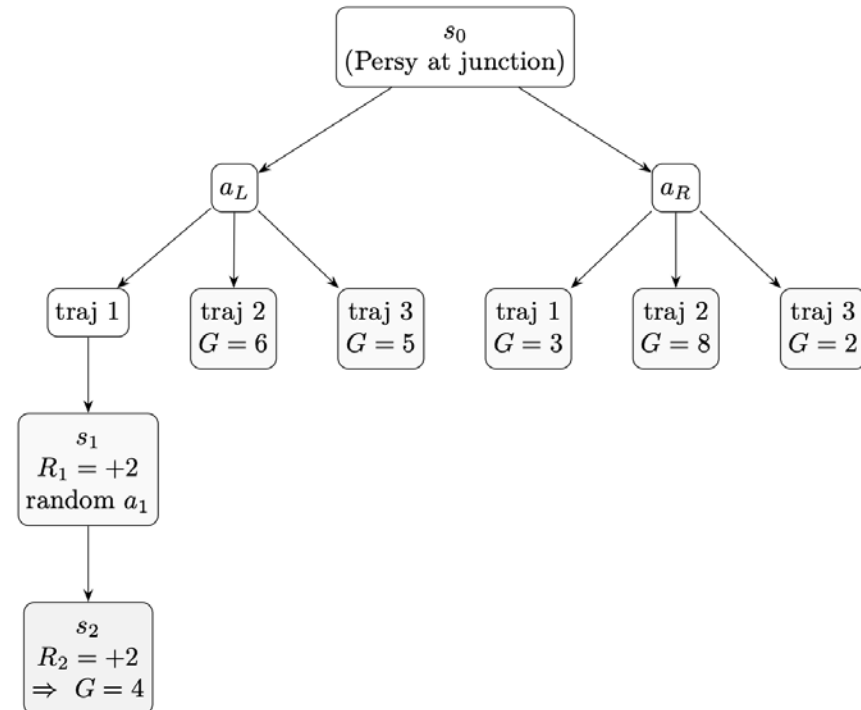
- Sampled returns (3 trajectories per action):

- a_L : $G_1 = 4, G_2 = 6, G_3 = 5$
 $\Rightarrow \hat{Q}^\pi(s_0, a_L) = \frac{4 + 6 + 5}{3} = 5.0$
- a_R : $G_1 = 3, G_2 = 8, G_3 = 2$
 $\Rightarrow \hat{Q}^\pi(s_0, a_R) = \frac{3 + 8 + 2}{3} = 4.33$

- Choose

- $a^* = \arg \max_a \hat{Q}^\pi(s_0, a) = a_L$.

Decision-Time and Demo-Based Planning



First action fixed (why?) → Random rollout actions

rollout policy π applies to actions after the first

$$\hat{Q}_{H=2}^\pi(s_0, a_L) = \frac{4 + 6 + 5}{3} = 5.0, \quad \hat{Q}_{H=2}^\pi(s_0, a_R) = \frac{3 + 8 + 2}{3} = 4.33.$$

$$\hat{Q}^\pi(s_0, a) = \frac{1}{N} \sum_{i=1}^N G_i \quad \Rightarrow \quad \hat{Q}^\pi(s_0, a_L) = 5.0, \quad \hat{Q}^\pi(s_0, a_R) = 4.33, \quad a^* = a_L.$$

Rollout: Strengths and Limitations

- Strengths

- Simple: simulate and average returns
- Effective: improves over rollout policy
- Flexible: works even with random policy
- Parallelizable: independent simulations

- Limitations

- High computation per decision
- Accuracy depends on rollout policy quality
- Limited by decision-time constraints



From Rollout to Monte Carlo Tree Search

- **Rollout:**
 - Simulate many trajectories from current state
 - Average returns for each action
 - Choose best action (one-step policy improvement)
- **Monte Carlo Tree Search (MCTS):**
 - Extends rollout with tree expansion and guided sampling
 - Balances exploration vs exploitation (e.g., Upper Confidence Bound)
 - Reuses rollout statistics across decisions
 - Enabled breakthroughs (e.g., AlphaGo)
- **Key Insight:**
 - MCTS = Rollouts + Smart Tree Search → scalable decision-time planning



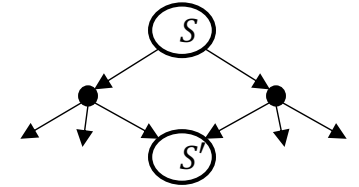
Monte Carlo Tree Search

Online search with simulation

Monte Carlo Tree Search

- Combines tree search with Monte Carlo rollouts
- Repeatedly simulate trials from the root state s_0 :
 - **Selection**: follow tree policy (explore vs exploit)
 - **Expansion & Simulation**: add new state s , rollout $\rightarrow$ sampled return G_t
 - **Backup**: propagate results back up the tree
- **Return**:
 - $G_t = R(S_t, A_t, S_{t+1}) + \gamma G_{t+1}$: additive discounted rewards from time t
- **Anytime policy**:
 - Stop when time is up, choose best root action
- **Example (tic-tac-toe)**:
 - simulate games from each move, expand promising ones, average returns
 $\rightarrow$ estimate utility of each move.

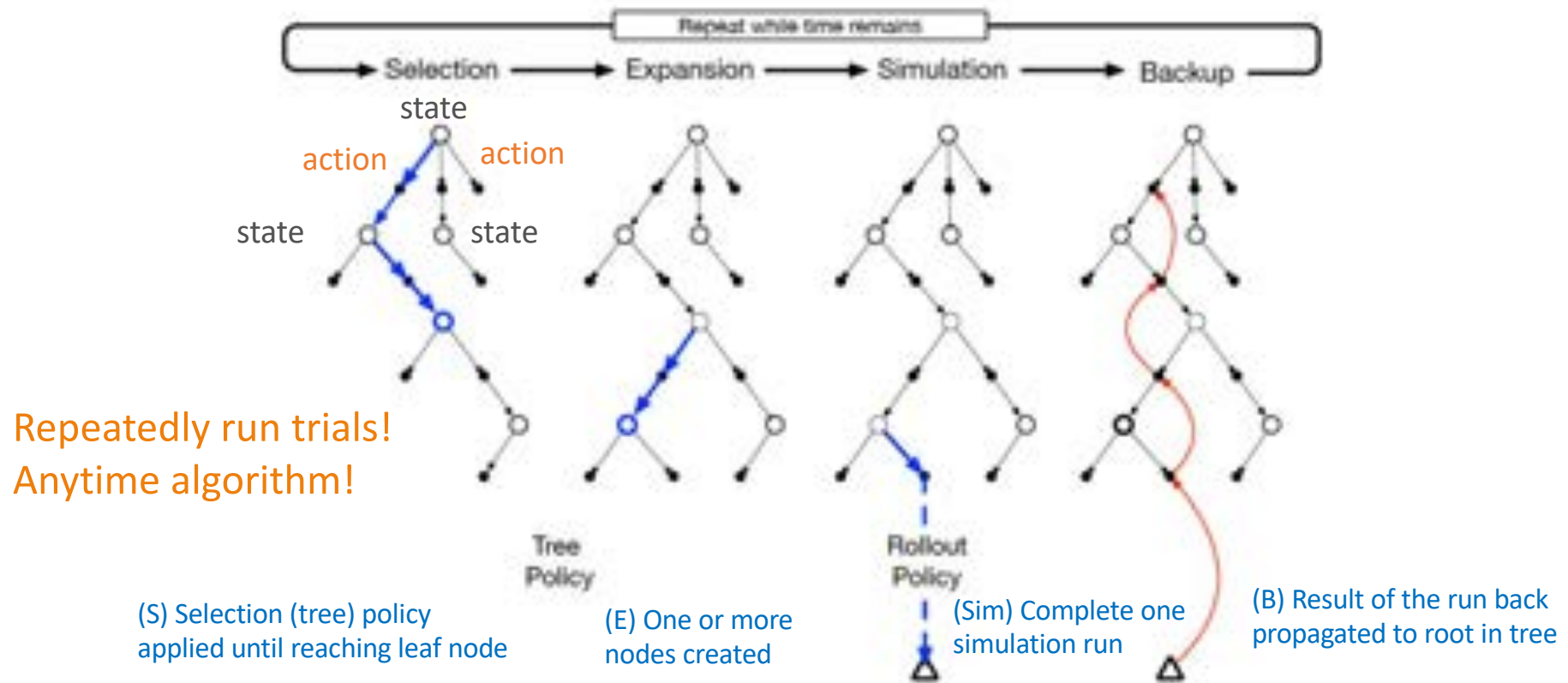
MCTS as Online Search in MDPs



- Nodes: correspond to states s
- Actions: chosen by selection (tree policy) (exploration vs exploitation)
- Transitions: sample next state s' via $P(s'|s, a)$
- Estimates: MCTS approximates utility by averaging sampled returns:
 - $\hat{Q}(s, a)$: average sampled returns G_t for action a at state s
 - $\hat{U}(s)$: average sampled returns G_t for state s
- Backup: returns propagate upward along trajectory
 - $G_t(s) = R(s, a, s') + \gamma G_{t+1}(s')$ G_t : additive discounted rewards from time t
- At the root s_0 : choose the best action

$$a^* = \arg \max_a \hat{Q}(s_0, a)$$

Monte Carlo Tree Search



Source: SB Figure 8.10

Upper Confidence Bounds applied to Trees (UCT)

Explores less-visited actions, converges with enough samples

- UCT Formula: Selection (tree) policy of UCT1 [2] algorithm at node n :

$$\pi_{UCT}(n) = \underset{a}{\operatorname{argmax}} \left(\hat{Q}(s, a) + c \sqrt{\frac{\ln(N(s))}{N(s, a)}} \right)$$

Exploitation Exploration

- Where:

- $\hat{Q}(s, a)$: estimated utility from averaged returns
- $N(s)$: number of times state s has been visited
- $N(s, a)$: number of times action a was chosen at s
- c : constant to balance exploration vs exploitation
- Exploitation**: $\hat{Q}(s, a)$ —average utility of s that starts with action a
- Exploration**: Term with $N(s, a)$ in denominator —encourages exploring less-visited actions.

- Properties

- With enough simulations, MCTS+UCT estimates converge: $\hat{Q}(s, a) \rightarrow Q^*(s, a)$, $\hat{U}(s) \rightarrow U(s)$
- In practice: strong performance in games & planning though worst-case efficiency can be poor

¹Levente Kocsis and Csaba Szepesvari. "Bandit based monte-carlo planning". In: European conference on machine learning. Springer. 2006, pp. 282-293.

¹Coquelin, P.-A. and R. Munos, *Bandit algorithms for tree search*, in *UAI*. 2007, AUAI Press: Vancouver, BC, Canada. p. 67–74.

Notes: Monte Carlo Tree Search Algorithm

function MONTE-CARLO-TREE-SEARCH($state$) **returns** *an action*

$tree \leftarrow \text{NODE}(state)$

while IS-TIME-REMAINING() **do**

$leaf \leftarrow \text{SELECT}(tree)$

$child \leftarrow \text{EXPAND}(leaf)$

$result \leftarrow \text{SIMULATE}(child)$

$\text{BACK-PROPAGATE}(result, child)$

return the move in $\text{ACTIONS}(state)$ whose node has highest number of playouts

Highest utility or win percentage

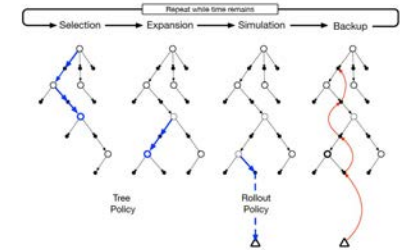


Figure 5.11 The Monte Carlo tree search algorithm. A game tree, $tree$, is initialized, and then we repeat a cycle of SELECT / EXPAND / SIMULATE / BACK-PROPAGATE until we run out of time, and return the move that led to the node with the highest number of playouts.

Example: Tic-Tac-Toe

- Scenario

- State: X to move, partial board (3 empty cells: TR, ML, BR)
- Goal: choose best move by planning ahead using Monte Carlo Tree Search (MCTS)

- MCTS Steps

- Selection — pick a leaf using UCB scores
- Expansion — add the node to the tree
- Simulation (rollout) — play random to terminal
- Backpropagation — update N, W, Q values up the tree

- UCT formula

- $$UCB = Q + c \sqrt{\frac{\ln N_{\text{parent}}}{N_{\text{child}}}}, c = \sqrt{2}$$

X	O	TR
ML	X	O
		BR

Rollout Progress (Iteration Snapshots)

- Iteration 1

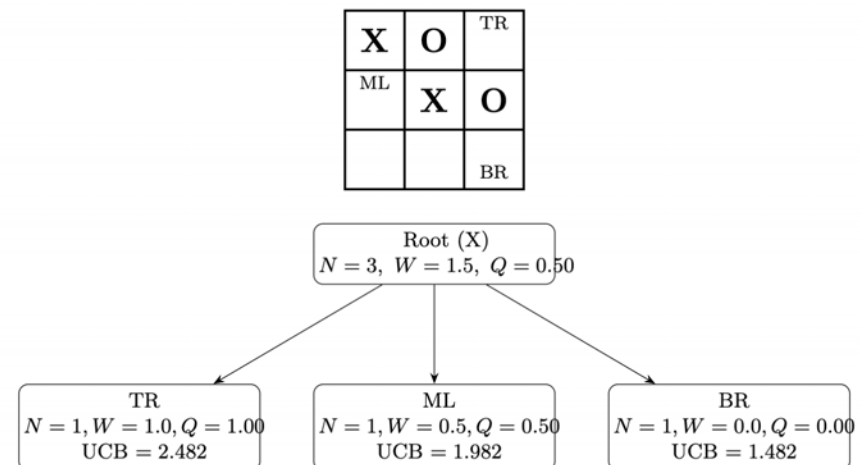
- Expand TR → rollout result: X win (1.0)
- Root: $N = 1, W = 1, Q = 1.00$

- Iteration 2

- Expand ML → rollout result: draw (0.5)
- Root: $N = 2, W = 1.5, Q = 0.75$
- UCB bonus $c\sqrt{\ln 2} \approx 1.177$
- $UCB_{TR} = 2.177, UCB_{ML} = 1.677$

- Iteration 3

- Expand BR → rollout result: loss (0.0)
- Root: $N = 3, W = 1.5, Q = 0.50$
- UCB bonus $c\sqrt{\ln 3} \approx 1.482$
- $UCB_{TR} = 2.482, UCB_{ML} = 1.982, UCB_{BR} = 1.482$



Example Calculations

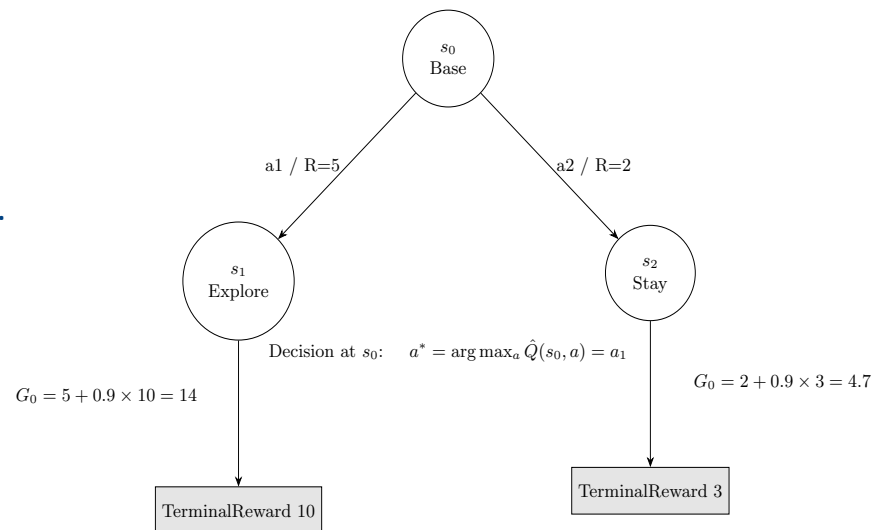
Iteration	Action expanded	Rollout result	Root (N, W, Q) after	Child updates (N, W, Q)	UCBs for next selection (parent N)
$0 \rightarrow 1$	TR (top-right)	Win (1.0)	$(1, 1, 1.00)$	TR $(1, 1, 1.00)$	Unvisited: $+\infty$; TR: $1.00 + \sqrt{2} \sqrt{\ln 1 / 1} = 1.00$ (parent $N = 1$)
$1 \rightarrow 2$	ML (middle-left)	Draw (0.5)	$(2, 1.5, 0.75)$	ML $(1, 0.5, 0.50)$	TR: $1 + 1.177 = 2.177$; ML: $0.5 + 1.177 = 1.677$; Unvisited: $+\infty$ (parent $N = 2$)
$2 \rightarrow 3$	BR (bottom-right)	Loss (0.0)	$(3, 1.5, 0.50)$	BR $(1, 0, 0.00)$	TR: $1 + 1.482 = 2.482$; ML: $0.5 + 1.482 = 1.982$; BR: $0 + 1.482 = 1.482$ (parent $N = 3$)

Example: Persy the Rover Adventures

- Persy is at base s_0 with two choices:
 - Explore (a_1) $\rightarrow$ reward 5, then terminal reward 10
 - Stay (a_2) $\rightarrow$ reward 2, then terminal reward 3
 - Discount factor: $\gamma = 0.9$
- Selection: At root s_0 , choose action using tree policy
- Expansion: Add child node if action not fully explored.
 - Example: Expand “Explore” $\rightarrow$ new node s_1 .
- Simulation: Rollout (sample trajectory) from new node.
 - Explore: $G_0 = 5 + 0.9 \times 10 = 14$
 - Stay: $G_0 = 2 + 0.9 \times 3 = 4.7$
- Backup: Propagate values back to root.
 - Root updates: $\hat{Q}(s_0, a_1) = 14$, $\hat{Q}(s_0, a_2) = 4.7$
- Decision: Best root action: Explore (a_1)

$$\text{Selection via UCT: } \max_a \hat{Q}(s, a) + c \sqrt{\frac{\ln N(s)}{N(s, a)}}$$

(UCT balances exploration vs exploitation).



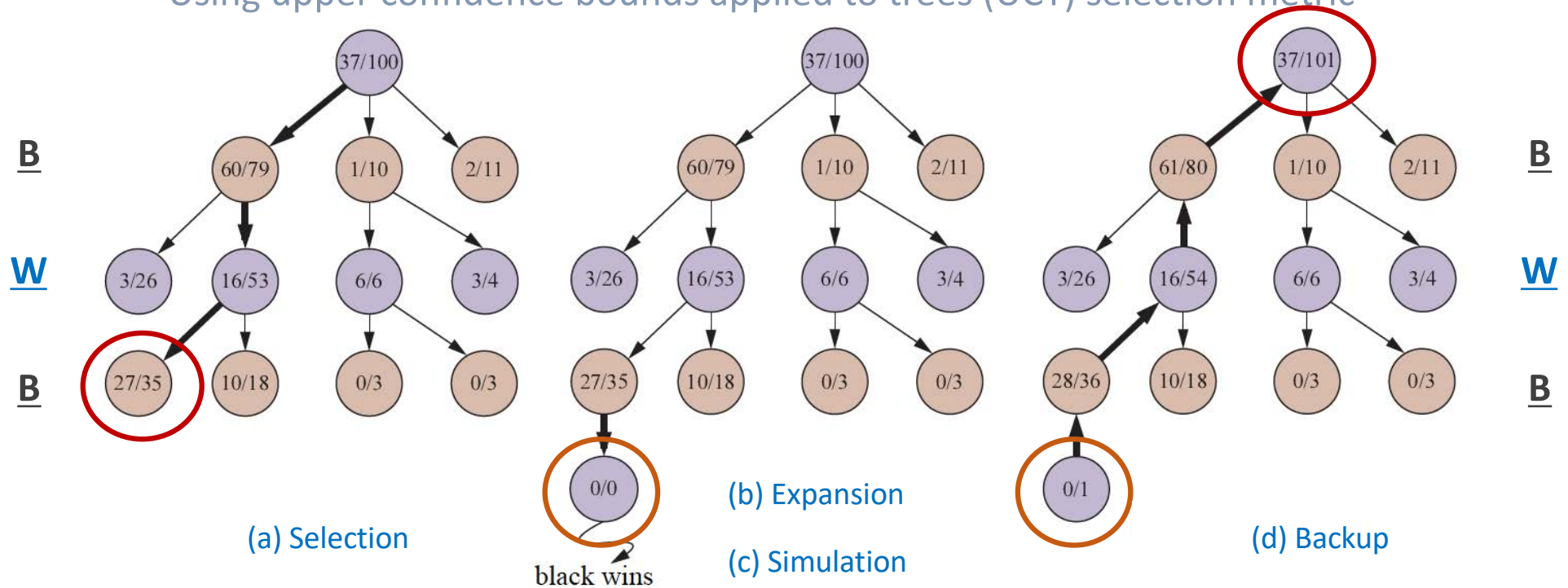


Exercise

- So far, we have left action selection at a node undefined.
- Q: How should we select an action at a node n ?
 - Assume a deterministic Markov decision process (MDP):
 - $P(s'|s, a)$ is deterministic.
 - Each node n corresponds to a state s .
- What if we use a deterministic function $f(s)$ to pick the action at each node (except the root)?
 - Then $A(s) = f(s)$ is just a fixed policy.
 - If we also use $f(s)$ as the rollout policy at the leaves, this becomes plain rollout, offering no improvement over rollout.
- Better approach:
 - At each node, explore actions first, then gradually shift to more greedy (exploit) choices , e.g., using:
 - Upper Confidence Bound (UCB1)
 - ϵ -greedy or softmax selection.

Example: Monte Carlo Tree Search in Game

Using upper confidence bounds applied to trees (UCT) selection metric



AlphaGo Zero



- Go complexity:
 - $\sim 10^{170}$ states, branching factor 361
- Solution: Deep neural net with two heads
 - Value: state value estimate
 - Policy: move probabilities
- Training:
 - AlphaGo Lee: expert games + self-play
 - AlphaGo Zero: only self-play, defeated AlphaGo Lee 100–0
- Game setting:
 - Zero-sum, turn-taking; terminal reward +1 / -1

- MCTS with UCT variant:

$$\pi_{UCT}(s) = \arg \max_a (\hat{Q}(s, a) + cP(s, a) \sqrt{\frac{\sum_b N(s, b)}{1 + N(s, a)}})$$

- Uses policy head priors
- Value head evaluates leaves (no rollout)
- Approximate policy iteration:
 - MCTS = policy improvement
 - Self-play + supervised learning = policy evaluation

David Silver et al. "Mastering the game of Go without human knowledge". In: Nature 550.7676 (2017), p. 354.
Decision-Time and Demo-Based Planning

Notes: MCTS in MDPs vs in Games

Aspect	MCTS in MDPs	MCTS in Games
Setting	Single-agent planning under uncertainty	Two-player, zero-sum, turn-taking
Return G_t	Additive discounted rewards	Win/loss outcome (+1 / -1)
Utility Estimate	$\hat{U}(s) = \text{avg of sampled returns}$	Expected win rate for current player
UCT / PUCT	Guides exploration–exploitation in MDP tree	Guides move search with policy priors (PUCT in AlphaGo Zero)
Leaf Evaluation	Rollouts or heuristic estimates	Value network (no rollouts)
Backup	Propagate discounted rewards	Propagate win probabilities (alternate turns)



Summary: Decision-Time Planning

- **Core Idea:**
 - Plan at runtime by simulating possible futures from the current state, instead of solving the full MDP offline.
- **When Effective:**
 - Large branching factors
 - Deep search trees
 - Simulator/model available
 - Limited real-time budget
- **Key Techniques:**
 - Online search: build shallow trees each step
 - Monte Carlo Tree Search (MCTS): combine tree search + rollouts
 - Upper Confidence Bound (UCT): balances exploration vs exploitation



Homework

- Readings
 - Online algorithms, MCTS:
 - [RN] 17.2.4, 5.4
 - [SB] 8.10, 8.11
 - MCTS and the UCT algorithm:
 - Guillaume Chaslot et al. “Monte-Carlo Tree Search: A New Framework for Game AI”. In: AIIDE. 2008
 - Kocsis, L. and C. Szepesvári, Bandit based Monte-Carlo planning, in Proceedings of the 17th European conference on Machine Learning. 2006, Springer-Verlag: Berlin, Germany. p. 282–293.
 - Coquelin, P.-A. and R. Munos, Bandit algorithms for tree search, in Proceedings of the Twenty-Third Conference on Uncertainty in Artificial Intelligence. 2007, AUAI Press: Vancouver, BC, Canada. p. 67–74
 - Levinson, R. (2024). Monte Carlo Tree Search for Integrated Planning, Learning, and Execution in Nondeterministic Python. In Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS). <https://icaps24.icaps-conference.org/program/workshops/prl-papers/13.pdf>
 - AlphaGo Zero:
 - David Silver et al. Mastering the game of Go without human knowledge". In: Nature 550.7676 (2017), p. 354.
- Decision-Time and Demo-Based Planning



Short Videos to Learn MCTS

- Visualizing MCTS
 - Clear step-by-step view of selection → expansion → simulation → backprop
 - [Watch on YouTube](#) (by Dave Dyer)
- Monte Carlo Tree Search Explained Visually
 - Concise visual animation of how MCTS builds and updates the search tree
 - [Watch on YouTube](#) (by Arxiv Insights)
- How AlphaZero Uses MCTS
 - Shows how neural networks + MCTS work together in modern planning
 - [Watch on YouTube](#) (by DeepMind)
 - David Silver et al. Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm, Science, 362(6419), 2018 [Read the paper](#)
- (Optional) AlphaGo Zero (precursor to AlphaZero)
 - [Intro video](#)
 - David Silver et al. Mastering the game of Go without human knowledge. Nature 550.7676 (2017), p. 354. [Read the paper](#)



Demonstration-Based Planning

Offline Learning from (Expert) Demonstrations



Demonstration-Based Planning

- Why demonstrations?
 - Rewards unknown or hard to specify
 - Exploration unsafe or costly
 - Experts available to demonstrate good behavior
- Apprenticeship Learning:
 - Learn from expert demonstrations
- Two approaches:
 - Imitation Learning (IL): learn policy directly
 - Inverse Reinforcement Learning (IRL): infer reward, then derive policy

Apprenticeship Learning

- Assuming observable environment, learning from examples of expert behavior
- Imitation Learning (IL)
 - Direct policy learning: Learns $\pi(s)$ directly from D – a set of expert demonstrations.
 - Supervised learning paradigm: No explicit modeling of environment or reward function.

$$\pi(s) = \operatorname{argmin}_{\pi} \sum_i \ell(\pi(s_i), a_i)$$

where ℓ is the loss function between predicted actions $\pi(s_i)$, and expert actions a_i .

- Inverse Reinforcement Learning (IRL):
 - Reward function recovery: Attempts to infer the expert's reward function $R(s, a)$.
 - Policy derivation: Once $R(s, a)$ is known, derive the optimal policy π^* by solving:

$$\pi^* = \operatorname{argmax}_{\pi} E_{\pi} [\sum_t \gamma^t R(s, a)]$$

where γ is the discount factor.

Decision-Time and Demo-Based
Planning

Typically, in imitation learning, loss function ℓ is chosen as the negative log-likelihood (NLL) of the expert action given the state under the policy π :
 $\ell(\pi(s_i), a_i) = -\log \pi(a_i | s_i)$



Imitation Learning

Imitation Learning (IL)

- What is Imitation Learning (IL)?
 - Learn a policy $\pi(a|s)$ by mimicking expert behavior
 - Use expert state–action pairs (s_i, a_i) as training data
 - Treat actions as labels $\rightarrow$ a supervised learning problem

- Objective:

$$\pi^* = \arg \min_{\pi} \sum_{(s_i, a_i) \in D} \ell(\pi(s_i), a_i) = \arg \min_{\pi} \sum_i \ell(\pi(s_i), a_i)$$

Behavioral Cloning (BC)

- Main idea:
 - Train a stochastic policy $\pi_{\theta}(a|s)$ to maximize expert action likelihood
- Pros: Simple, fast
- Cons: Covariate shift → compounding errors
 - Training on expert states only
 - During execution, agent drifts to unseen states → more mistakes

Notes: Behavioral Cloning

- Objective

- Train a stochastic policy $\pi_{\theta}(a|s)$ parameterized by θ to imitate expert behavior.
- Learn from a dataset $D = \{(s_i, a_i)\}$ of expert state-action pairs.

- Optimization Problem

- Maximize the likelihood of expert actions under the policy:

$$\theta^* = \arg \max_{\theta} \prod_{(s_i, a_i) \in D} \pi_{\theta}(a_i | s_i) \quad \text{or} \quad \theta^* = \arg \max_{\theta} \sum_{(s_i, a_i) \in D} \log \pi_{\theta}(a_i | s_i)$$

- Define the loss as the negative log-likelihood:

$$\ell(\pi_{\theta}(s_i), a_i) = -\log \pi_{\theta}(a_i | s_i)$$

- Equivalent minimization problem

$$\theta^* = \arg \min_{\theta} \sum_{(s_i, a_i) \in D} \ell(\pi_{\theta}(s_i), a_i)$$

Pitfalls: Why Supervised Learning Fails

- Supervised Learning Assumption:
 - Training and test data come from the same distribution (i.i.d.)
 - Imitation Learning Reality:
 - Learner's actions change future states → breaks i.i.d. assumption

Key Challenges:

- Distribution Mismatch:
 - States visited during execution differ from expert demonstrations
- Compounding Errors:
 - Early mistakes lead to drift into unfamiliar states → more errors
- Limited Generalization:
 - Learner may fail to recover in unseen or rare states

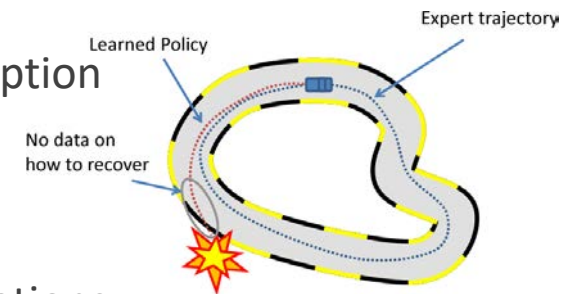


Figure from DAGGER Thesis¹

¹Stephane Ross. "Interactive learning for sequential decisions and predictions". PhD Thesis. Carnegie Mellon University, 2013.
Decision-Time and Demo-Based Planning

DAgger (Dataset Aggregation)

- Iterative improvement over BC:
 1. Train policy on expert demos
 2. Execute current policy → collect visited states
 3. Ask expert to label those states
 4. Aggregate new data and retrain
- Key ideas:
 - Trains on states the learner actually visits
 - Reduces distribution mismatch and improves robustness
- Variants:
 - SafeDAgger (safety), DART/SHIV (less expert input)
 - HG-DAgger (generalization), Agnostic IIL (handles imperfect experts)

Notes: Key Equations

- Dataset update rule

$$D_{i+1} = D_i \cup \{(s, a_E^*)\}$$

- s : states visited by the current learner policy π_i during rollout
- $a_E^* = \pi_E(s)$: expert's action for state s , queried from the expert policy π_E
- D_i : aggregated dataset up to iteration i

- Policy training objective

$$\pi_{i+1} = \operatorname{argmin}_{\pi} \sum_{(s_i, a_i) \in D_i} \ell(\pi(s_i), a_i)$$

Expert action

- ℓ : loss function between learner and expert actions. (e.g. cross-entropy between predicted and expert actions)

- Key benefits

- Incorporates states the learner actually visits
- Reduces distribution mismatch
- Improves robustness to compounding errors

Reference:

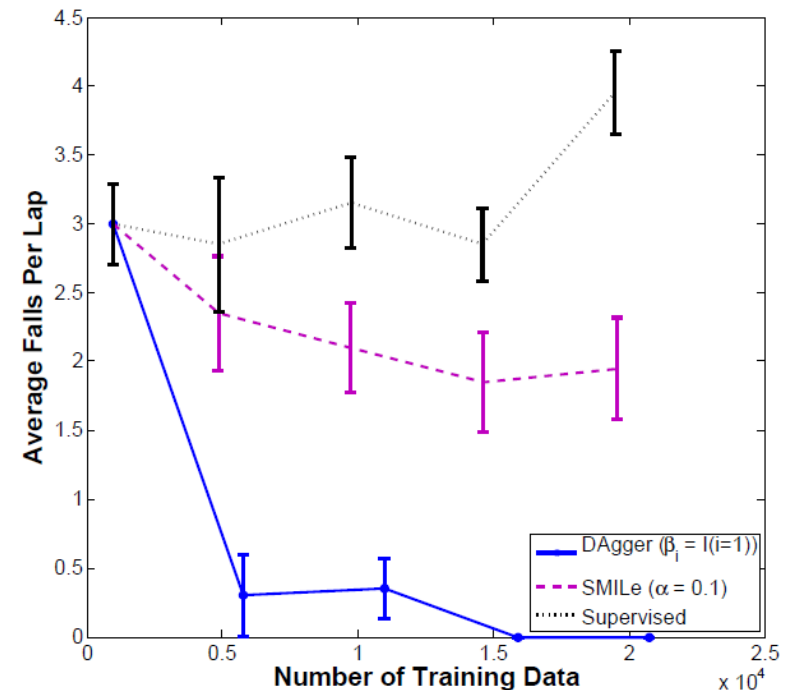
Ross, S., Gordon, G., & Bagnell, D. (2011). "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning." Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS), pp. 627–635.

Decision-Time and Demo-Based Planning

Videos source: <https://www.youtube.com/watch?v=V00npNnWzSU> (released by the authors)

Dagger – Experiment

- Task:
 - Train a kart to steer from image input at fixed speed
 - Goal: complete laps without falling off
- Performance Metric:
 - Average number of falls per lap (lower is better)
- Method:
 - Expert controls kart via joystick to give demonstrations
- Results:
 - DAgger quickly reduces falls to near zero
 - Supervised degrades due to state distribution drift
 - SMILe improves moderately but slower than DAgger



Source: Ross, S., Gordon, G., & Bagnell, D. (2011). "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning." Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS), pp. 627–635.

Videos source: <https://www.youtube.com/watch?v=V00npNnWzSU> (released by the authors)

Notes: DAgger – Exploit the expert

Initialize $\mathcal{D} \leftarrow \emptyset$.

Expert policy

Initialize $\hat{\pi}_1$ to any policy in Π .

for $i = 1$ **to** N **do**

Let $\pi_i = \beta_i \pi^* + (1 - \beta_i) \hat{\pi}_i$.

- Use expert policy with some probability β to obtain the dataset
- Decaying β

Sample T -step trajectories using π_i .

Exploit the expert

Get dataset $\mathcal{D}_i = \{(s, \pi^*(s))\}$ of visited states by π_i and actions given by expert.

Aggregate datasets: $\mathcal{D} \leftarrow \mathcal{D} \cup \mathcal{D}_i$.

Train classifier $\hat{\pi}_{i+1}$ on $\mathcal{D}$.

end for

Return best $\hat{\pi}_i$ on validation.

Select best of the N policies

This method reduces distribution mismatch and improves robustness to errors during execution.



Summary: Imitation Learning

- **Goal:**
 - Learn behavior from expert actions (no rewards or optimality)
- **Pros:**
 - Simple, effective for automated tasks
- **Cons:**
 - Prone to compounding errors; limited to expert performance – no notion of reward or optimality
- **Main techniques:**
 - Behavioral cloning: supervised policy learning
 - DAgger: iterative expert corrections
 - GAIL: adversarial approach (policy vs discriminator)



Summary: Beyond Full Offline Planning and Reinforcement Learning

- **Decision-time planning**
 - Simulate forward at runtime from the current state
 - Build shallow search trees on demand
 - Avoid solving the full MDP or training a full RL policy offline
- **Demonstration-based planning**
 - Learn from expert behavior before runtime
 - Derive policies from demonstrations (e.g. Imitation Learning)
 - Reduces data and compute cost compared to full RL training
- **Shared Advantage**
 - More sample- and compute-efficient than full offline planning or fully trained RL
 - Practical for real-time and resource-limited settings



Human Factors and Human-Guided Planning

- Human input can improve planning efficiency
- But human decisions are affected by:
 - Cognitive biases
 - Judgmental heuristics
 - Risk-sensitive behavior (→ Prospect theory)
- Challenge:
 - Understanding how human factors shape decision-making
- Future lectures on human-guided decision making:
 - Game theory – multi-agent decision modeling
 - Model-based RL – integrating human priors
 - LFM-assisted planning – with human guidance



Homework

- Readings:
 - RN 22.6 (*Optional - Apprenticeship Learning*)
- References on Imitation Learning
 - Agnostic Interactive Imitation Learning: Li, Y., & Zhang, C. (2024). "Agnostic Interactive Imitation Learning: New Theory and Practical Algorithms." Proceedings of the Forty-First International Conference on Machine Learning (ICML)
 - Gavenski, N., O. Rodrigues and M. Luck (2024). Imitation learning: A survey of learning methods, environments and metrics. arXiv preprint arXiv:2404.19456 <http://arxiv.org/abs/2404.19456>
 - Duan, Y., Andrychowicz, M., Stadie, B., Ho, J., Schneider, J., Sutskever, I., Abbeel, P., & Zaremba, W. (2017). "One-Shot Imitation Learning." NeurIPS, 30, 1087–1098.
 - Hussein, A., M. M. Gaber, E. Elyan and C. Jayne (2017). Imitation learning: A survey of learning methods. ACM Comput. Surv. 50(2): Article 21. 10.1145/3054912 <https://doi.org/10.1145/3054912> NUS: <https://dl-acm-org.libproxy1.nus.edu.sg/doi/pdf/10.1145%2F3054912>
 - Ross, S. (2013). "Interactive Learning for Sequential Decisions and Predictions." Proceedings of the 16th International Conference on Artificial Intelligence and Statistics (AISTATS), pp. 689–697.
 - Ross, S., Gordon, G., & Bagnell, D. (2011). "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning." AISTATS, 627–635.
 - Ho, J. and S. Ermon (2016). Generative adversarial imitation learning. Advances in Neural Information Processing Systems. 4572–4580. https://proceedings.neurips.cc/paper_files/paper/2016/file/cc7e2b878868cbae992d1fb743995d8f-Paper.pdf



Additional References

References on Imitation Learning based on Dagger: (Chronological)

1. DAgger: Ross, S., Gordon, G., & Bagnell, D. (2011). "A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning." Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS), 627–635.
2. Interactive Learning for Sequential Decisions and Predictions: Ross, S. (2013). Interactive learning for sequential decisions and predictions. PhD Thesis, Carnegie Mellon University.
3. SafeDAgger: Zhang, J., & Cho, K. (2016). "Query-Efficient Imitation Learning for End-to-End Autonomous Driving." arXiv preprint arXiv:1605.06450.
4. SHIV: Laskey, M., Lee, J., Fox, R., Dragan, A. D., & Goldberg, K. (2016). "SHIV: Reducing Supervisor Burden in DAgger Using Support Vectors for Efficient Learning from Demonstrations in High Dimensional State Spaces." IEEE ICRA, 462–469.
5. DART: Laskey, M., Staszak, S., Hsieh, W. Y., Mahler, J., Pokorny, F. T., & Goldberg, K. (2017). "DART: Noise Injection for Robust Imitation Learning." CoRL, 143–156.
6. Deeply AggreVaTeD: Sun, W., Venkatraman, A., Gordon, G. J., Boots, B., & Bagnell, J. A. (2017). "Deeply AggreVaTeD: Differentiable Imitation Learning for Sequential Prediction." ICML, 3309–3318.
7. End-to-End Imitation Learning: Codevilla, F., Müller, M., López, A., Koltun, V., & Dosovitskiy, A. (2018). "End-to-End Driving Via Conditional Imitation Learning." IEEE ICRA, 4693–4700.
8. HG-DAgger: Kelly, A., Sidrane, C., Dragan, A. D., & Goldberg, K. (2019). "HG-DAgger: Interactive Imitation Learning with Human Experts." IEEE ICRA, 8077–8083.

Some Imitation Learning Software

Framework	Maintainers	Key Algorithms	Focus	GitHub
robomimic	NVIDIA / Stanford (ARISE)	BC, BC-RNN, DAgger-style, GAIL	Robotics, multi-modal IL & offline RL	 robomimic
imitation	CHAI	BC, AIRL, GAIL, DAgger	Core IL + IRL on Gym tasks	 imitation
X-IL	Research consortium	BC, DAgger, GAIL-style	Large-scale, multi-modal IL (RoboCasa, LIBERO)	 X-IL
Stable Baselines3	DLR / Community	Backend for BC, DAgger, GAIL via imitation	General RL framework supporting IL	 SB3